Operating Systems Week 11 Lecture 11

Junaid Abdul Wahid

Previous

- Memory Management Background
- Contiguous Memory Allocation

Today Agenda

- Virtual Memory
- Demand Paging

Objectives

Define virtual memory and describe its benefits.
Illustrate how pages are loaded into memory using demand paging.

Apply the FIFO, optimal, and LRU page-replacement algorithms.
Describe the working set of a process, and explain how it is related to program locality.

Describe how different OS manages virtual memory.

Design a virtual memory manager simulation in the C programming language.

Background

Code needs to be in memory to execute, but entire program rarely used

- Error code, large data structures

The entire program code is not needed at the same time

Consider the ability to execute partially loaded program

- Program no longer constrained by limits of physical memory

- Each program takes less memory while running -> more programs run at the same time

- Less I/O needed to load or swap programs into memory -> each user program runs faster

Virtual memory

Virtual memory – separation of user logical memory from physical memory

- Only part of the program needs to be in memory for execution
- Logical address space can therefore be much larger than physical address space
- Allows address spaces to be shared by several processes
- Allows for more efficient process creation
- More programs running concurrently
- Less I/O needed to load or swap processes

"Virtual Memory allows programs to behave as if the computer has more memory than it physically has, by using disk storage to simulate extra RAM."

Virtual Memory

- A memory management technique that gives **an illusion** to each process that it has **its own large, continuous block of memory**, even if the **physical RAM is small** or shared between many processes.

- It **extends** the available memory by using a part of the hard disk (or SSD) as if it were RAM.

- Programs can run even if they are **larger than the available physical memory**!

Virtual memory (Cont.)

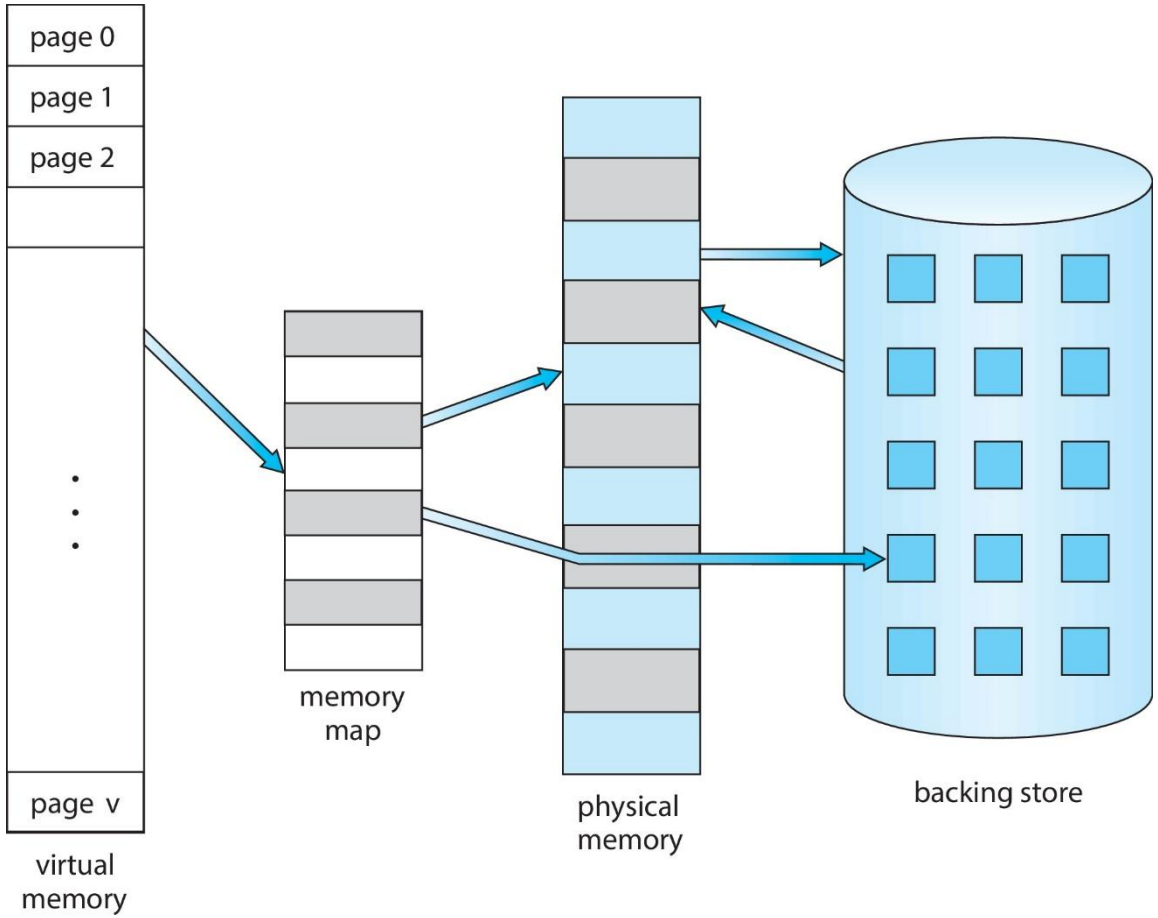
Virtual address space – logical view of how process is stored in memory

- Usually start at address 0, contiguous addresses until end of space
- Meanwhile, physical memory organized in page frames
- MMU must map logical to physical

Virtual memory can be implemented via:

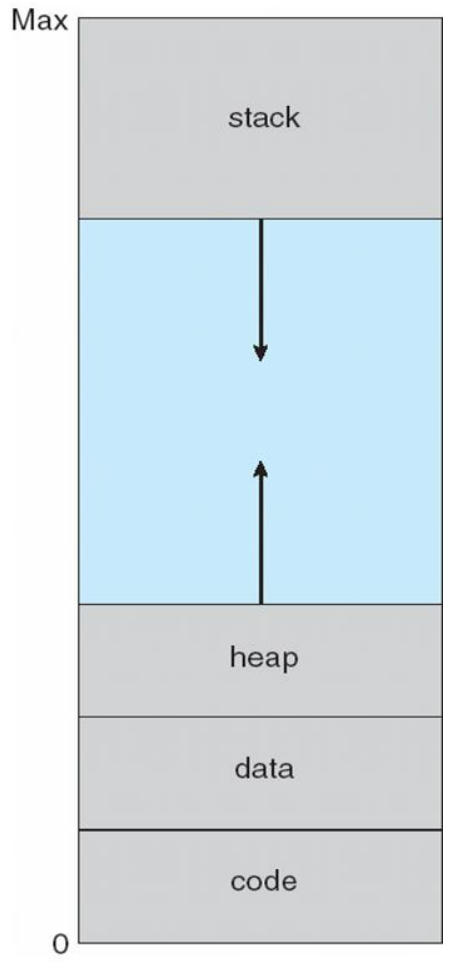
- Demand paging
- Demand segmentation

Virtual Memory That is Larger Than Physical Memory

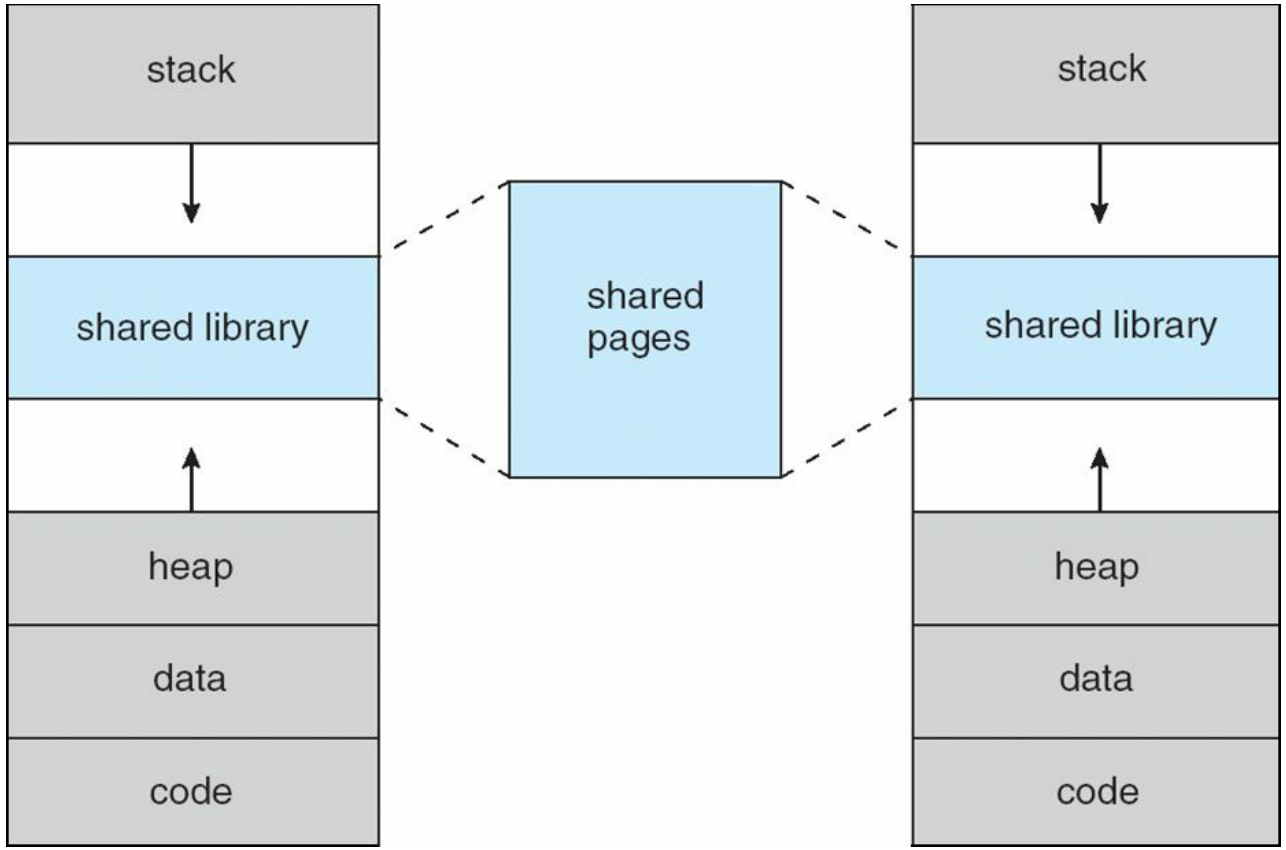


Virtual-address Space

- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
  - Maximizes address space use
  - Unused address space between the two is hole
    - No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc.
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during `fork()`, speeding process creation



Shared Library Using Virtual Memory



Why is it important

Reason	**Explanation**
Run larger programs	Programs bigger than physical RAM can run smoothly.
Multiprogramming	Many programs can run at the same time without interfering with each other.
Memory protection	Each process is isolated from others — no accidental crashes.
Simplified programming	Developers don’t need to manage memory complexity manually.

How it works

- Each process thinks it has its **own private memory** — this is the *virtual address space*.

- The OS and hardware (Memory Management Unit - **MMU**)
translate virtual addresses to **physical addresses**.

- If a needed part of a program is **not yet in RAM**,
the OS **loads it from disk** (this is called **paging** or **swapping**).

- Only the **active parts** of programs are kept in RAM; inactive parts stay on disk.

Example Virtual Memory

- You have a **small desk** (RAM).
- But you are working on a **huge project** with **many books** (programs and data).
- You **keep only a few books** open on your desk (RAM) at a time.
- If you need another book, you **swap** it from the bookshelf (Disk) onto your desk.

Your desk space feels bigger than it actually is, because you manage what's active smartly — this is Virtual Memory!

Example Virtual Memory

Operating System	How Virtual Memory is Used
Windows	Uses a **Pagefile** (pagefile.sys) on disk to store memory pages not currently in RAM.
Linux	Uses **swap space** (swap partition or swap file) for virtual memory.
MacOS	Also uses swap files when RAM is full to avoid crashes.

When your computer becomes *slow* when opening many apps — it's because it is using virtual memory (swap) heavily, and disk is much slower than RAM.

Demand Paging

Demand paging is a memory management technique where **pages are loaded into RAM *only when they are needed*** — not in advance.

- At the start, **nothing** (or only a few pages) of the process is loaded into memory.
- When the program tries to access a page that is **not yet in RAM**, the operating system **pauses** the program, loads that page from **disk (secondary storage)** into RAM, and then **resumes** execution.

Demand Paging

Why we need it

Advantage	Explanation
Faster process startup	Don't load everything at once — load only needed pages.
Saves memory	Only the parts of programs that are actually used occupy RAM.
Enables larger programs	Programs can be bigger than physical memory — unused parts stay on disk until needed.

Demand Paging

How Demand Paging Works (Step-by-Step)

1.Process starts execution → Minimal or no pages are loaded initially.

2.Process tries to access a page → CPU issues a logical address.

3.Page not in RAM?

➔ *Page Fault* occurs.

4.OS handles the page fault:

1. Find the page in the disk (backing store).
2. Find a free frame in RAM (may need to evict another page if no free frame).
3. Load the requested page into RAM.
4. Update the page table to mark the page as "present."

5.Resume the process from where it left off.

Demand Paging example

Suppose you open **Microsoft Word**:
•Initially, only the parts needed for showing the splash screen and UI are loaded.
•You open a large document → Word starts pulling only needed parts of the file into memory, not the whole file at once.

Key Points To Remember
•Demand paging **loads pages only when needed**.
•It improves **efficiency** but may cause **page faults** (and slow down temporarily).
•It depends on **page replacement algorithms** to manage RAM efficiently.